

Capabilities — Proof-Based Security Testing

Shannon performs real, automated exploitation across web, transport, DNS, cloud and network layers — and only records a finding when it is confirmed by a benign proof signal. It then correlates findings into attack chains, demonstrates real-world impact, and can watch the attack surface continuously. Zero false positives by construction.

36

PROOF-BASED CLASSES

5

LAYERS (WEB · TLS · DNS · CLOUD · NETWORK)

0

FALSE POSITIVES (BY CONSTRUCTION)

Exploit → Impact → Chain → Defend → Report

EVERY CONFIRMED FINDING

✓ Proof engine — the truth

Deterministic probers send real crafted requests and confirm with a benign marker. If Shannon says **CONFIRMED**, it is real — this layer carries zero false positives.

◆ AI layer — the judgment

An LLM adversarially **judgets** findings and **proposes** app-specific tests — but a proposal counts only after the engine verifies it deterministically. AI can cut noise, never invent a finding.

Web application (25 classes)

Injection & execution

CLASS	WHAT SHANNON PROVES (THE MARKER)	SEVERITY
SQL injection	Error-based, boolean-blind, and time-based — DB error / TRUE≠FALSE / an injected sleep that scales with the delay.	Critical
SQLi auth-bypass	Actually logs in via ' OR '1'='1' -- (TRUE gets a session that FALSE + bad creds do not).	Critical
NoSQL injection	key[\$ne] vs key[\$eq] changes the query result set (MongoDB).	Critical
Command injection	Shell evaluates echo \$((7*13)) → 91.	Critical
Server-side template injection	{{9931*9817}} → 97492627.	Critical
Insecure deserialization	A benign pickle payload evaluates a high-entropy product.	Critical
XXE	The XML parser resolves an external entity to an out-of-band listener.	High
CRLF / response splitting	An injected %0d%0a creates an attacker-controlled response header.	High
Host header injection	An attacker Host reflects into a link/redirect (reset & cache poisoning).	Medium
XSS — reflected / stored / DOM	Injected JavaScript actually EXECUTES in a real browser (stored = on a different page; DOM = a client-side sink the server never reflected).	High

Access control & authentication

CLASS	WHAT SHANNON PROVES	SEVERITY
BOLA / BFLA (2-session)	User A reads User B's private data; a low-priv user invokes an admin-only function.	Critical
IDOR (single-session)	Distinct per-record secrets returned for different IDs without authorization.	High
Auth testing	Default creds that actually log in, missing rate-limiting, username enumeration (determinism-gated).	Critical
JWT / token forgery	A weak-secret token is accepted; a forged admin token unlocks gated content.	High
CSRF	State-changing form with no token and a SameSite=None session cookie.	Medium

Server-side, request, client-side & API

CLASS	WHAT SHANNON PROVES	SEVERITY
SSRF	Server fetches an attacker URL (out-of-band callback).	High
Path traversal / LFI	Reads a system file it shouldn't (/etc/passwd).	Critical
Open redirect / CORS	Redirect to an attacker host; arbitrary Origin reflected with credentials.	High
Mass assignment	An unexpected privileged field is bound (proven selectively — a control field is not).	High
API excessive data exposure	A JSON API leaks a sensitive field (password hash / token / PII) with a real value.	High
GraphQL abuse	Introspection enabled; field suggestions leak schema; batching enabled.	Medium
Prompt injection (LLM)	The model follows an injected instruction (nonce oracle).	High

Information disclosure & configuration

CLASS	WHAT SHANNON PROVES	SEVERITY
Secrets exposure	A sensitive file (.env, .git, backups) is served, by content signature.	High
Verbose errors	Malformed input leaks a stack trace / debug page / server file path.	Medium
Security headers / templates	Missing CSP/HSTS/etc.; data-driven exposure checks (phpinfo, git HEAD, actuator).	Low

Beyond the web — transport, DNS, cloud & network (4 modules)

MODULE	WHAT SHANNON PROVES (AN OBSERVED FACT)	SEVERITY
TLS/SSL configuration	Deprecated TLS 1.0/1.1, expired / self-signed / hostname-mismatch / weak-key certificates — from the real handshake.	Medium
Subdomain takeover	A dangling CNAME to a known service (S3/GitHub Pages/Heroku/Azure/...) serving its "unclaimed" fingerprint.	Critical
Cloud storage exposure	A bucket the site references (yours) that is publicly listable — S3 / GCS / Azure.	High
Exposed services (opt-in)	Unauthenticated Redis / Memcached / Elasticsearch / anonymous FTP on the verified host.	Critical

Platform intelligence (advanced layers)

LAYER	WHAT IT ADDS	
Attack-path chaining	Correlates CONFIRMED findings into attack narratives (SSRF → cloud takeover; leaked secrets + exposed datastore → data breach; auth-bypass → full admin control) — deterministic, proof-based.	Story
Impact / post-exploitation	Safely DEMONSTRATES blast radius — SQLi pulls the DB version (readable), command injection shows the OS user (root?). Benign markers only.	Proof of impact
Continuous monitoring	Differs each scan against a baseline and alerts on NEW exposures (new subdomain, cert expiring, newly-open port, fresh vuln). Always-on attack-surface watch.	Watch
AI reasoning	An LLM proposes app-specific tests + business-logic hypotheses; only what the engine VERIFIES deterministically becomes a finding — the rest are labeled leads. AI cuts noise, never invents.	Augment

How every confirmed finding is handled

Exploit (real request + benign marker) → **Prove impact** (safely demonstrate the blast radius) → **Chain** (correlate into an attack path) → **Defend** (a detection rule + a concrete code fix, and for payload classes a live proxy that proves the attack is now blocked with a 403) → **Report** (CVSS 3.1 + vector, mapped to OWASP WSTG/ASVS, OWASP Top 10, CWE, MITRE ATT&CK, PCI DSS 4.0 and ISO 27001, with evidence and a sign-off block).

Straight talk. The **proof engine is real and deterministic** — a CONFIRMED finding is backed by a real request and a benign marker, not a guess, across every layer. The **AI layer is judgment** — it cuts noise and proposes ideas, but can never invent a finding; its unverified suggestions are labeled leads. Network scanning is **opt-in** and limited to the verified host. Coverage is focused, not exhaustive: deep business-logic and HTTP request smuggling remain manual. Shannon is not a replacement for a full manual penetration test, and a report becomes a **certified** pentest only once a certified human (OSCP / CREST) reviews and signs it. Accuracy you can trust, honestly stated.